

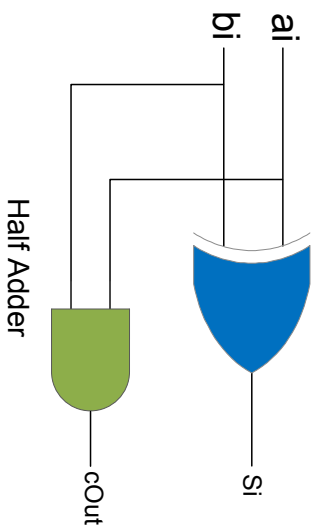
Adders

- Used in addition, subtraction, multiplication and division
- Speed of a signal processing or communication system ASIC depends heavily on these functional units

Basic Building Blocks

Lecture 5

Half Adder using Data Flow modeling



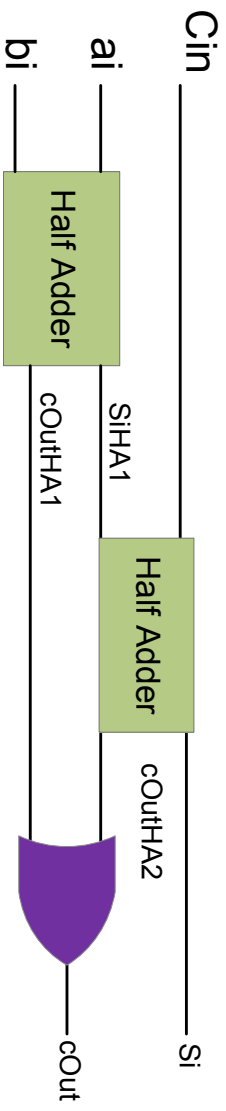
$$C_i = a_i \cdot b_i$$

$$S_i = a_i \oplus b_i$$

Half Adder using Data Flow modeling

```
module HALF_ADDER(ai, bi, si, cout);  
  input ai, bi;  
  output si, cout;  
  
  // data flow modeling  
  assign {cout,si} = ai + bi;  
  
endmodule
```

Full Adder



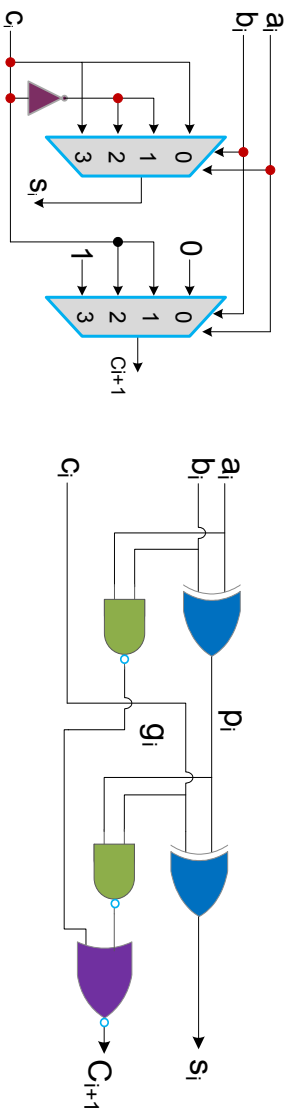
$$c_i = a_i \oplus b_i + a_i b_i$$

$$s_i = a_i \oplus b_i \oplus c_i$$

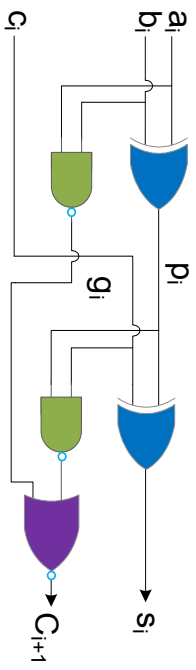
Full Adder

Truth Table					
x	y	z	CS		
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	1	1	1

Contd ...

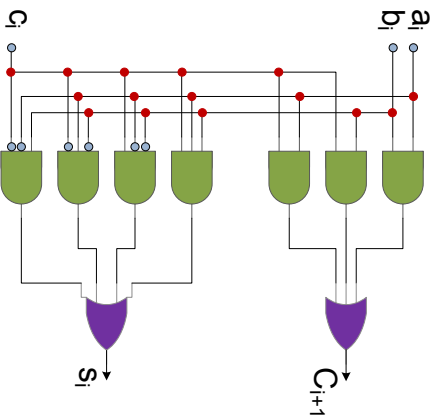


(c)

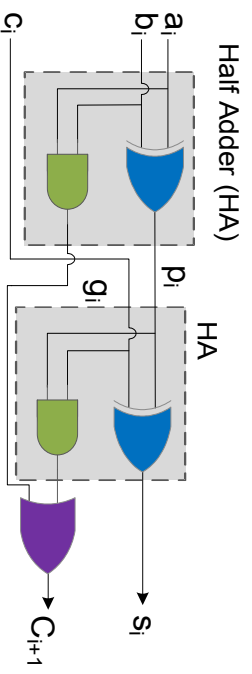


(d)

Gate-level design options for a full adder



(a)



(b)

Full Adder Using Data Flow Modeling

```
module FULL_ADDDER(ai,bi,cin,si,cout);  
    input ai,bi;  
    input cin;  
    output si,cout;
```

// through data flow level of abstraction

```
assign {cout,si} = ai + bi + cin;  
endmodule
```

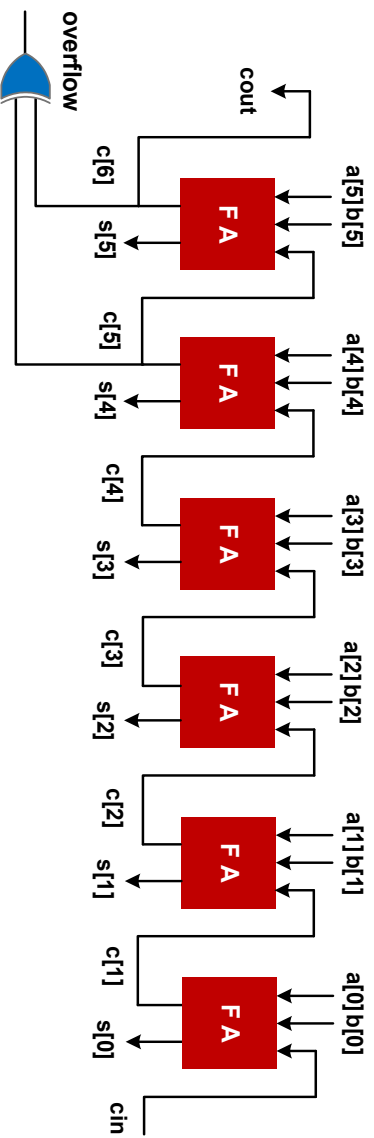
Full Adder: Implementation in Verilog

```
module FULL_ADDDER(ai,bi,cin,si,cout);  
    input ai,bi;  
    input cin;  
    output si,cout;  
    wire SiHA1,CoutHA1,CoutHA2;
```

```
    HALF_ADDDER HA1(ai,bi,SiHA1,CoutHA1); // instance HA1  
    HALF_ADDDER HA2(SiHA1,cin,si,Cout); //instance HA2  
    Or (cout,CoutHA1,CoutHA2); // using or gate primitive
```

```
endmodule
```

Ripple Carry Adder



Full Adder Using Data Flow Modeling

```
module FULL_ADDER(ai,bi,cin,si,cout);  
    input ai,bi;  
    input cin;  
    output si,cout;  
  
    // through data flow level of abstraction  
    assign {cout,si} = ai + bi + cin;  
endmodule
```

RCA: Dataflow modeling

Six bit ripple carry adder through data flow modeling

```
// SIX BIT FULL ADDER ;
module fulladder_6bit(s,cout,a,b,cin);

    output cout;
    output [5:0] s;
    input  [5:0] a,b;
    input  cin;
    reg [5:0] s,c;
    reg cout;
    always@(a or b or cin)
    begin
        {c[0],s[0]}= a[0] + b[0] + cin;
        for(i=1; i<6; i=i+1)
            {c[i],s[i]}= a[i] + b[i] + c[i-1]; // through data flow modeling.
        cout = c[5];
    end
endmodule
```

```
module ripple_carry_adder #(parameter W=16)
    (input      clk,
     input  [W-1:0] a, b,
     input      cin,
     output reg [W-1:0] s_r,
     output reg      cout_r);

    wire [W-1:0] s;
    wire      cout;
    reg  [W-1:0] a_r, b_r;
    reg          cin_r;

    assign {cout,s} = a_r + b_r + cin_r;

    always@(posedge clk)
    begin
        a_r<=a;
        b_r<=b;
        cin_r<=cin;
        s_r<=s;
        cout_r<= cout;
    end
endmodule
```

Group Carry and Group Propagate

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 c_1$$

$$\begin{aligned} &= g_1 + p_1 (g_0 + p_0 c_0) \\ &= g_1 + p_1 g_0 + p_0 p_1 c_0 \end{aligned}$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$$

$$\text{let } G_0 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$$

$$\text{and } P_0 = p_3 p_2 p_1 p_0$$

$$\text{we can write } c_4 = G_0 + P_0 c_0$$

⁴⁸Digital Design of Signal Processing Systems, John Wiley & Sons by Dr. Shoaib A. Khan

Carry Generate and Propagate Logic

$$g_i = a_i b_i$$

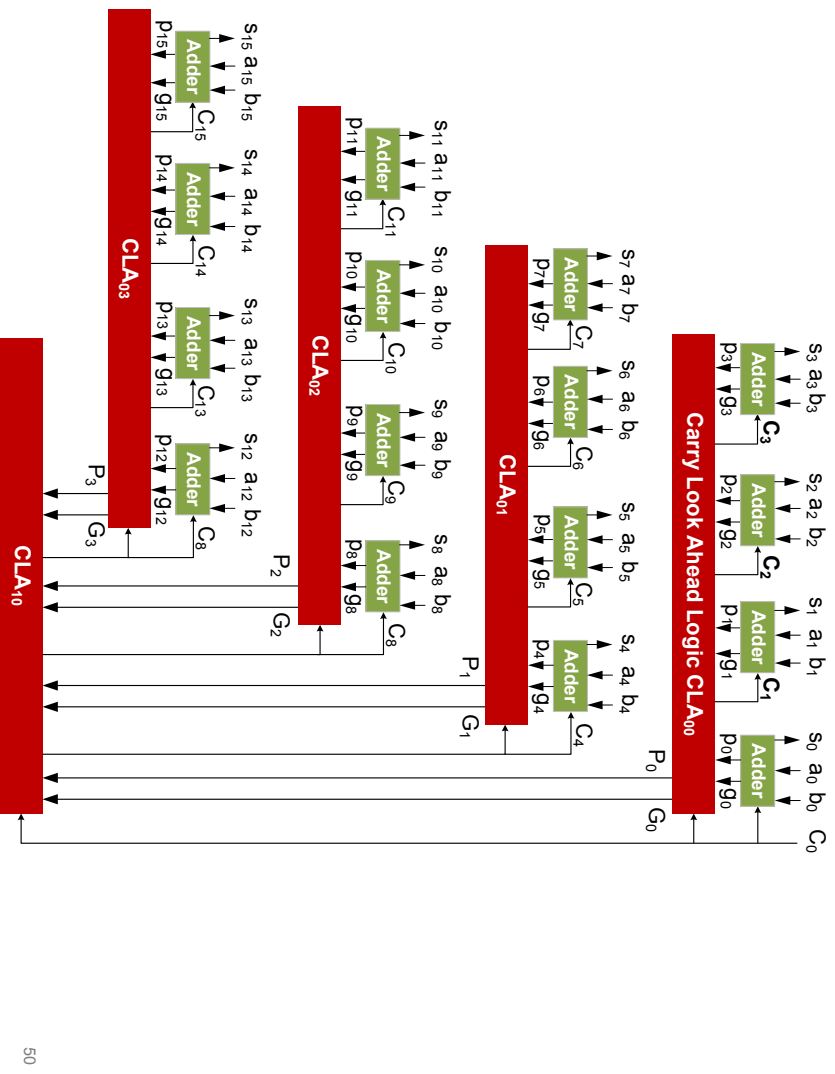
$$p_i = a_i \oplus b_i$$

$$c_{i+1} = g_i + p_i c_i$$

$$s_i = c_i \oplus p_i$$

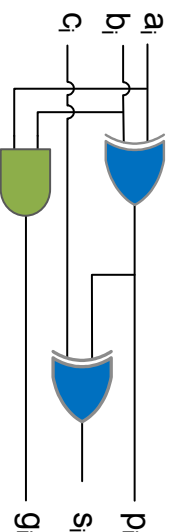
⁴⁷Digital Design of Signal Processing Systems, John Wiley & Sons by Dr. Shoaib A. Khan

A 16-bit carry look-ahead adder using two levels of CLA logic

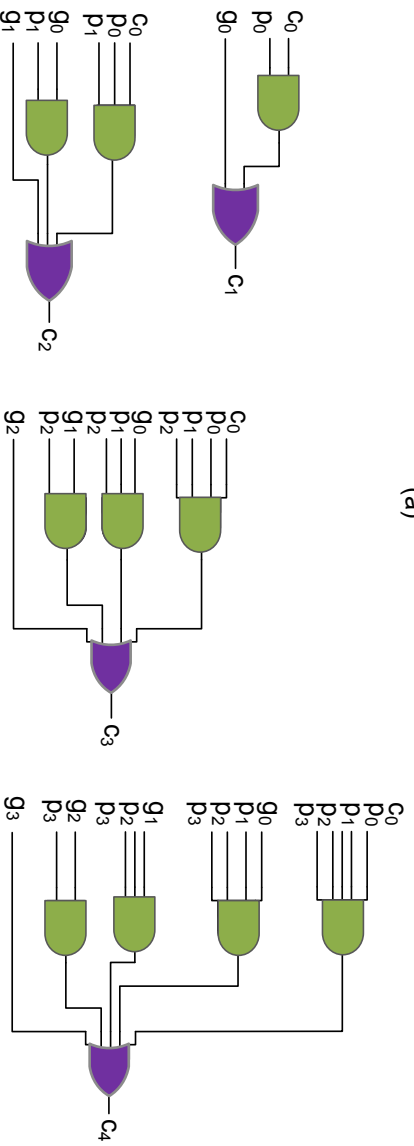


50

CLA logic for computing carries in two-Gate delay time

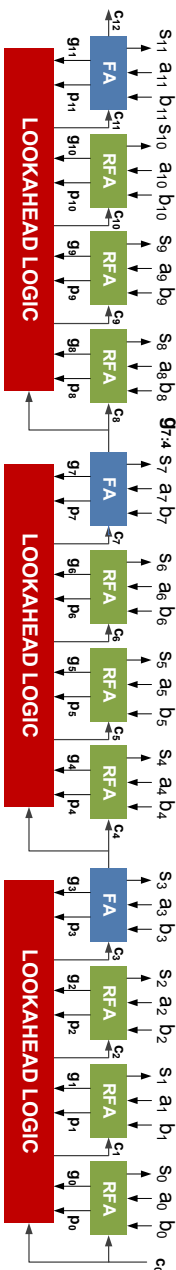


(a)

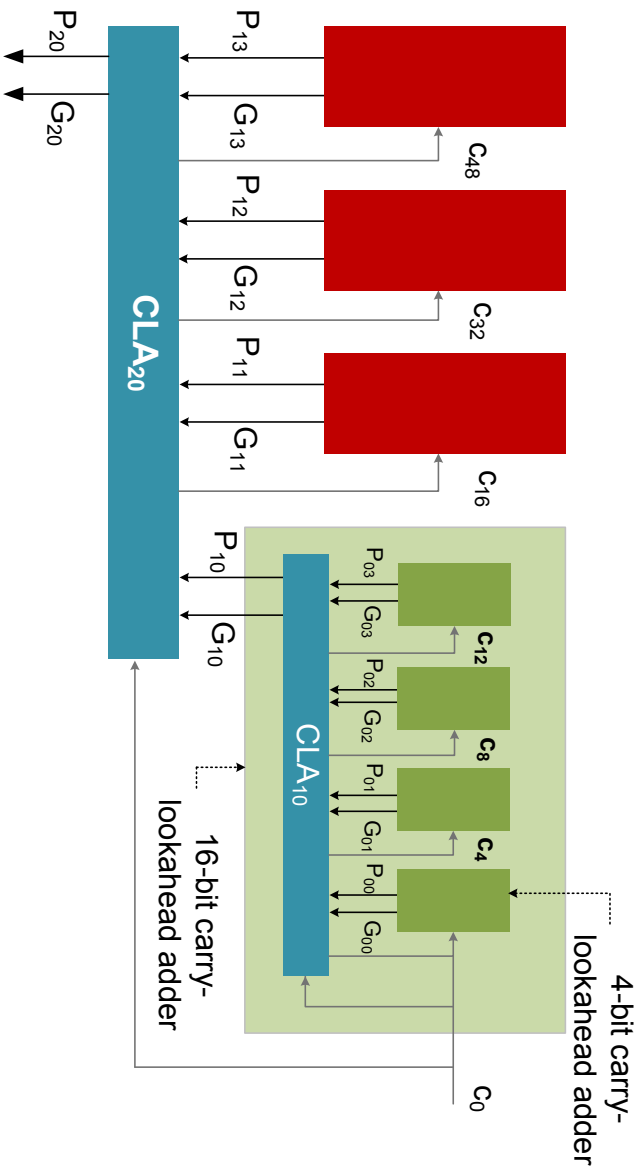


(b)

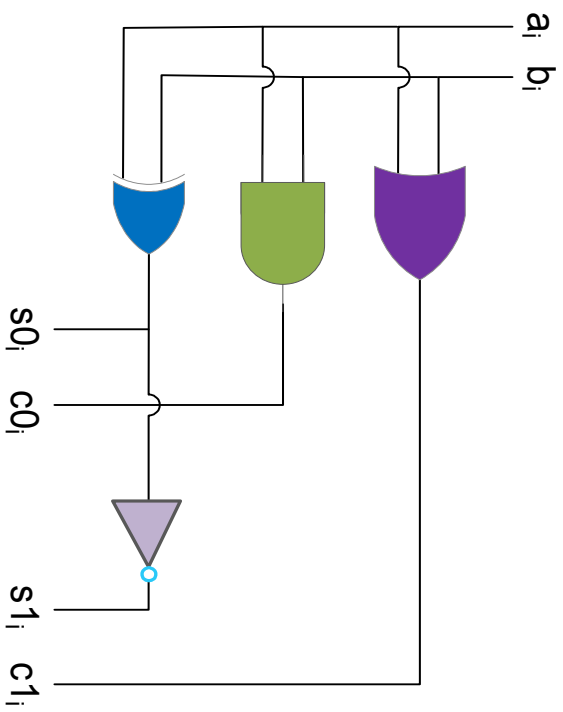
A 12-bit Hybrid Ripple Carry and Carry Look-ahead Adder



A 64-bit carry look-ahead adder using three levels of CLA logic



Conditional Cell (CC)



Example

$$s0_i = a_i \oplus b_i$$

$$s1_i = a_i \sim \oplus b_i$$

$$c0_i = a_i b_i$$

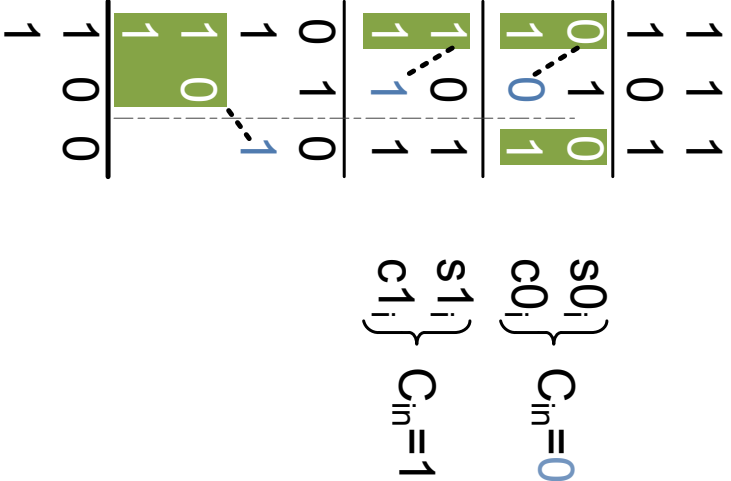
$$c1_i = a_i + b_i$$

Example: Conditional Sum Adder

		a _i	1	0	0	1	1	0	0	1	1	0	0	1	1	0	1
		b _i	0	0	1	1	0	1	1	0	1	1	0	1	0	1	0
		Group sum and block carry out															
Group width	Group carry-in																
		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	1	0	1	1	1	0	0	1	0	1	0	1	1	1
	1	0	0	0	1	0	0	0	0	1	1	0	0	0	1	0	0
2	0	1	0	0	0	1	1	1	1	1	0	0	1	0	0	1	1
	1	0	1	0	1	0	0	0	0	0	1	1	0	0	1	0	0
4	0	1	1	0	0	1	1	1	1	1	0	0	1	0	0	1	1
	1	0	1	0	1	0	0	0	0	1	0	1	0	0	1	1	1
8	0	1	1	0	0	1	1	1	1	1	1	0	1	0	0	1	1
	1	1	1	0	1	0	0	0	0								
16	0	1	1	0	1	0	0	0	0	1	0	1	0	0	0	1	1
	1																

Addition of three bit numbers using a conditional sum adder

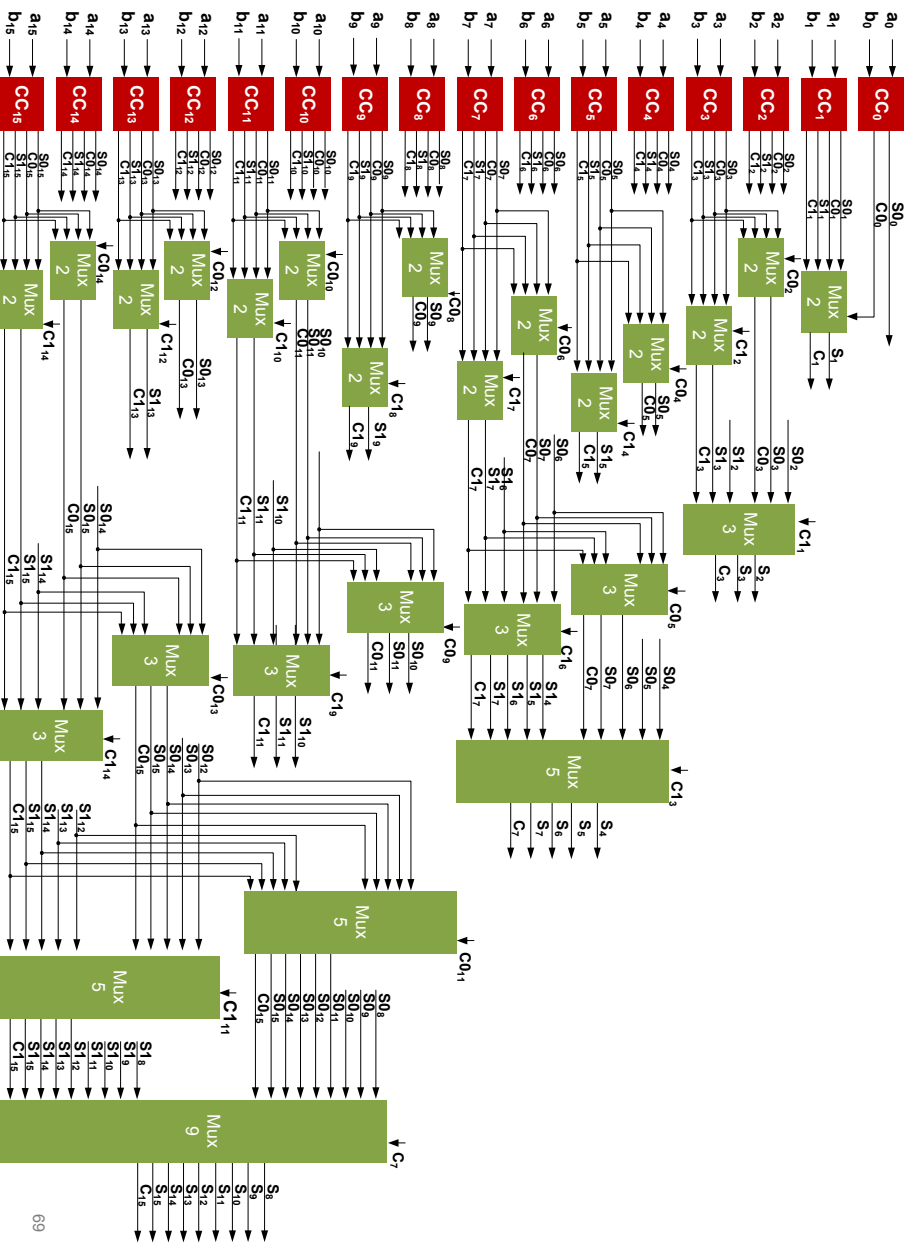
(Here we are assuming actual $c_{in}=0$)



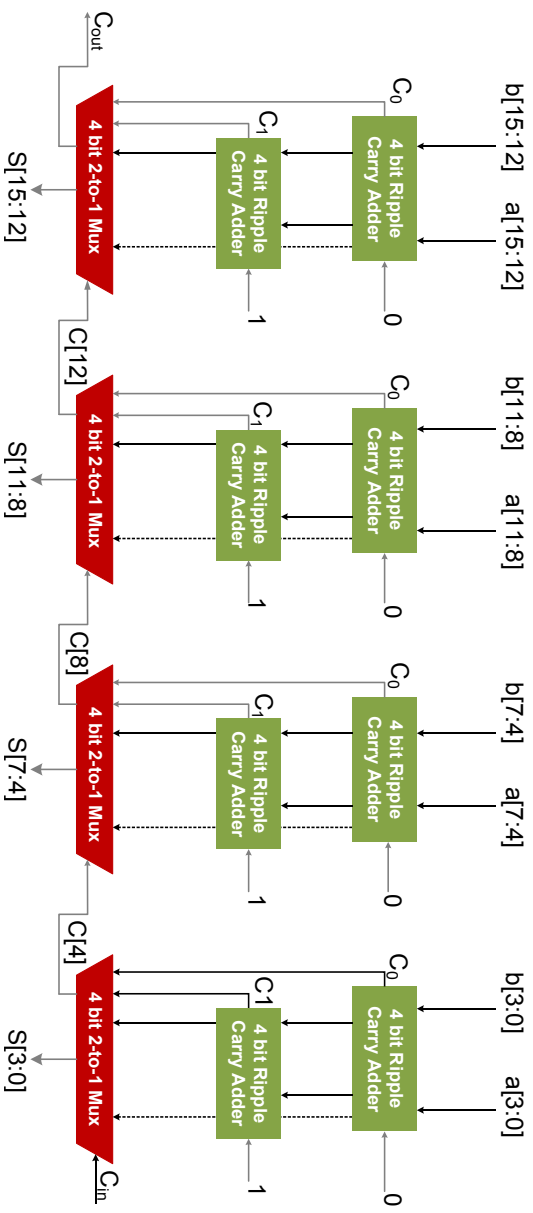
Hybrid Adder Designed

- Hybrids are obtained by combining elements of:
 - Ripple-carry adders
 - Carry-lookahead (generate-propagate) adders
 - Carry-skip adders
 - Carry-select adders
 - Conditional-sum adders
- You can obtain adders with
 - higher performance
 - greater cost-effectiveness
 - lower power consumption

A 16-bit Conditional Sum Adder



A 16-bit uniform-groups carry select adder



Example

$$s0_i = a_i \oplus b_i$$

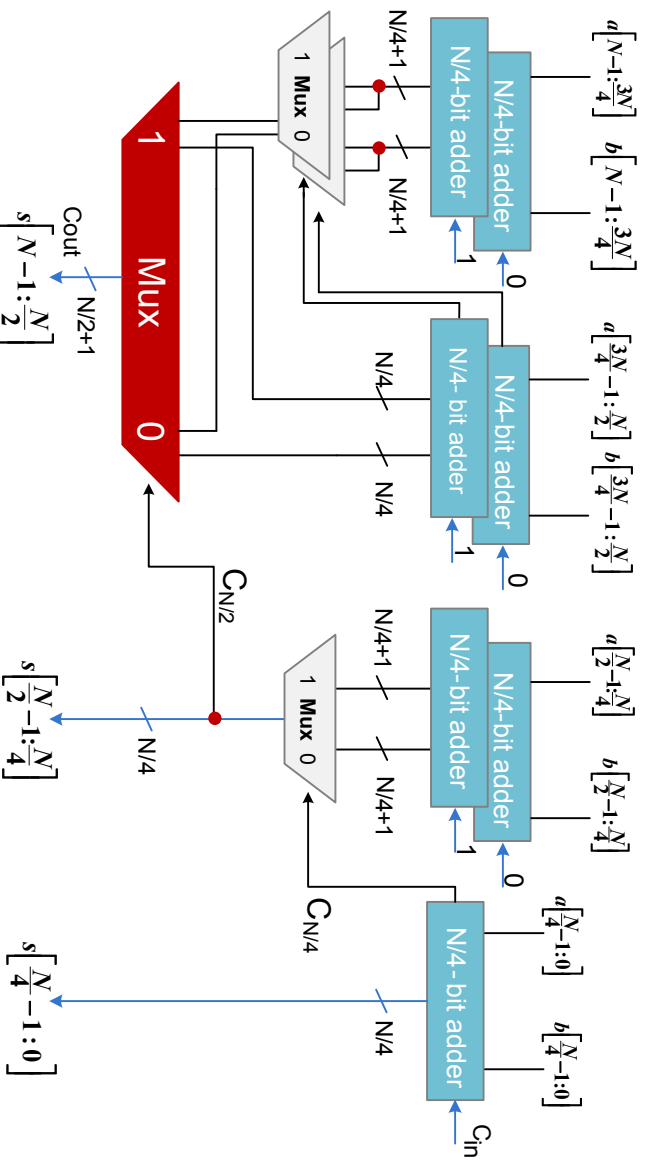
$$s1_i = a_i \sim \oplus b_i$$

$$c0_i = a_i b_i$$

$$c1_i = a_i + b_i$$

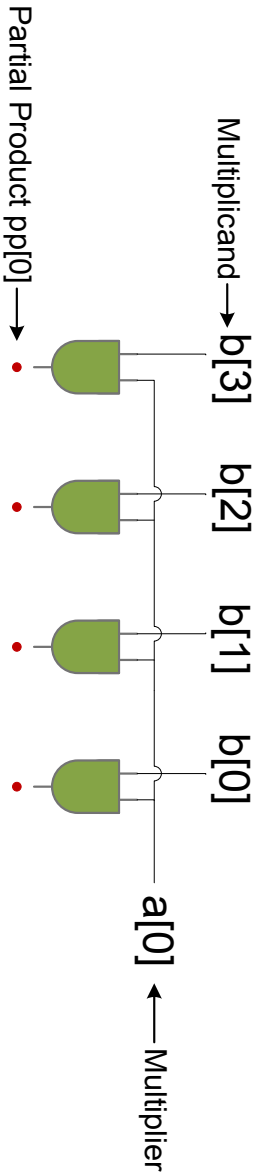
Carry Save Adders and Compressors

Hierarchical CSA



Dots are used to represent each bit of the partial product

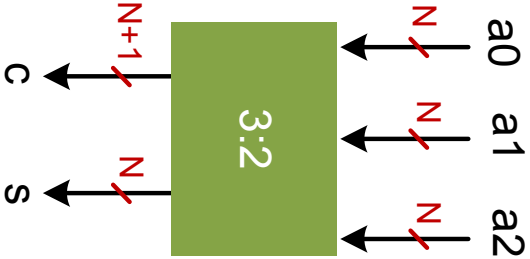
- Dot notation facilitates description of different reduction schemes
- Dots are used to represent each bit of the partial product



Carry Save Addition saves the carry at next bit location

- The CSA does not ripple any carry
- It has a delay of one FA
- The concept of CSA is effective in designing partial products compression/reduction logic

a0	=	0	0	1	0	1	1
a1	=	0	1	0	1	0	1
a2	=	1	1	1	1	0	1
s	=	1	0	0	0	1	1
c	=	0	1	1	1	0	1



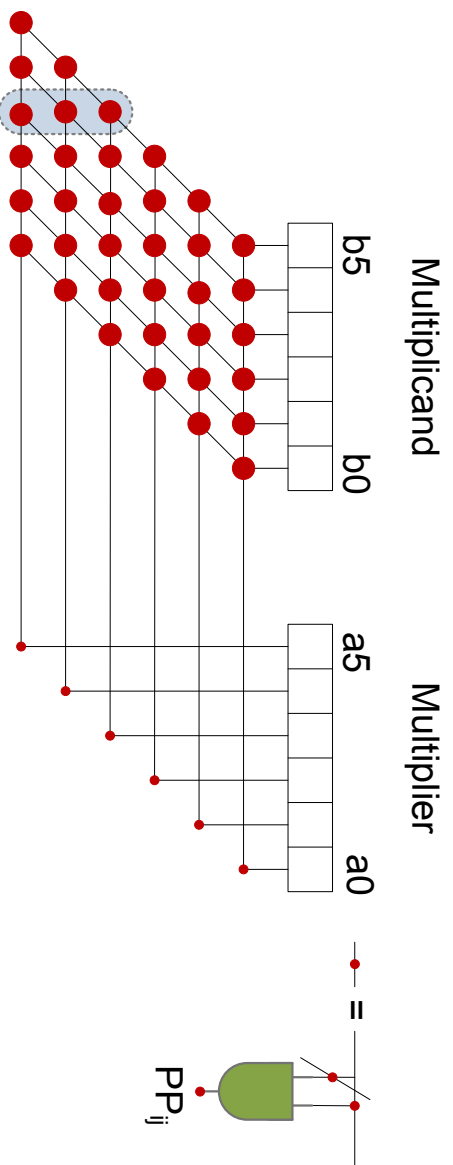
Three components of a multiplier

- N-bit inputs operands
- Partial Product Array Generation = N shifted binary numbers
- Partial Product Array Reduction= reduction to 2 binary numbers
- Final addition = 2N-bit final product

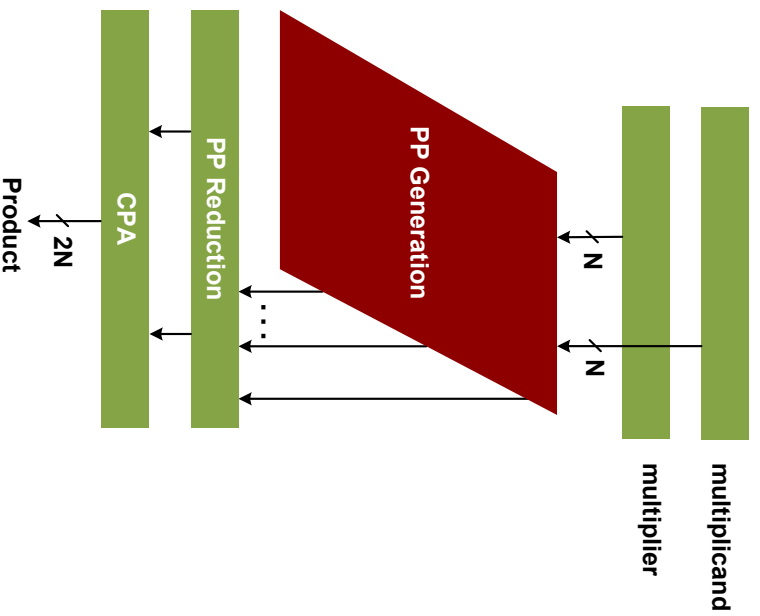
Parallel multiplier architecture

Designing Customized Multipliers

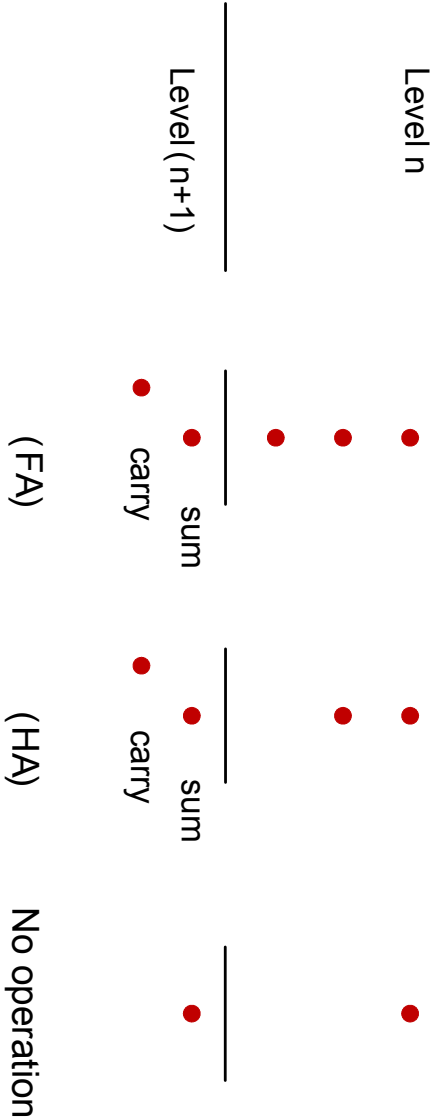
Partial Product Generation for a 6x6 Multiplier



Three components of a multiplier



Reducing number of dots in a column



Partial Product Generation Verilog Code

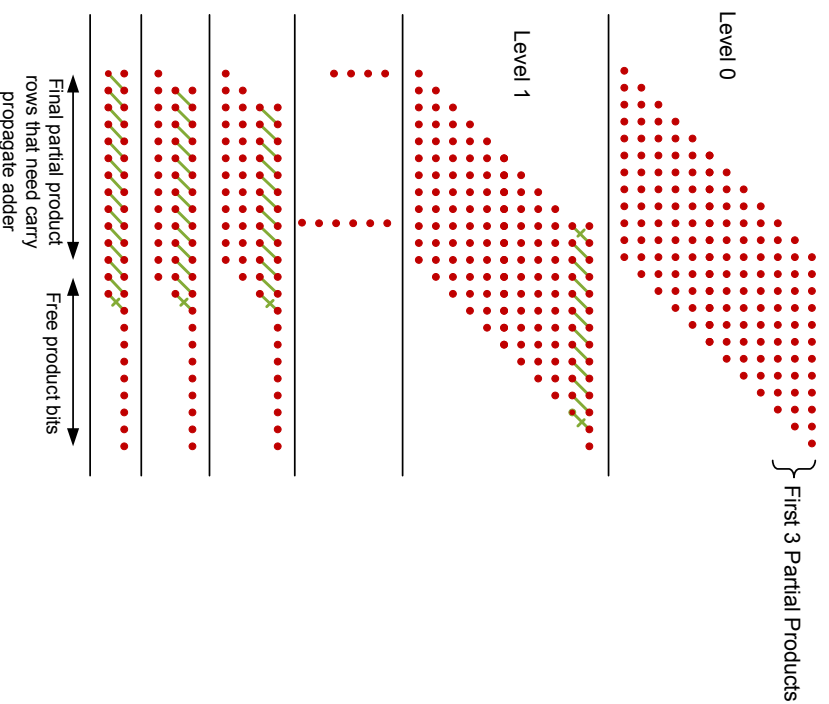
```
module multiplier (  
    input [5:0] a,b,  
    output [11:0] prod);  
  
    integer i;  
  
    reg [5:0] pp [0:5]; // 6 partial products  
  
    always@*  
    begin  
        for(i=0; i<6; i=i+1)  
            begin  
                pp[i] = b & {6{a[i]}};  
            end  
        end  
  
    assign prod = pp[0]+{pp[1],1'b0}+{pp[2],2'b0}  
        +{pp[3],3'b0}+{pp[4],4'b0}+{pp[5],5'b0};  
  
endmodule
```

Partial Products Reduction Schemes

- Carry Save Reduction Scheme
- Dual Carry Save Reduction Scheme
- Wallace Tree Reduction Scheme
- Dadda Tree Reduction Scheme

-
- Three dots are shown
 - Each symbolizes a partial product
 - Using FA reduces these to two bits
 - One has the weight of 2^0 (sum)
 - The other has the weight of 2^1 (carry)
 - This type of reduction is known as 3 to 2 reduction or carry saves reduction
 - The two dots are reduced to 2 using a HA

PP reduction for a 12x12 Multiplier using Carry Save Reduction Scheme



90

12x12 Carry Save Reduction Scheme

- Considers three rows at a time
- Take first three rows use CSA to reduce them to two
- Iteratively take two layers from previous reduction and a new from PP layer and reduce them to two using a CSA
- Finally produces two layers
- Also produces free product bits
- The two layers are added using any CPA

Dual Carry Save Reduction

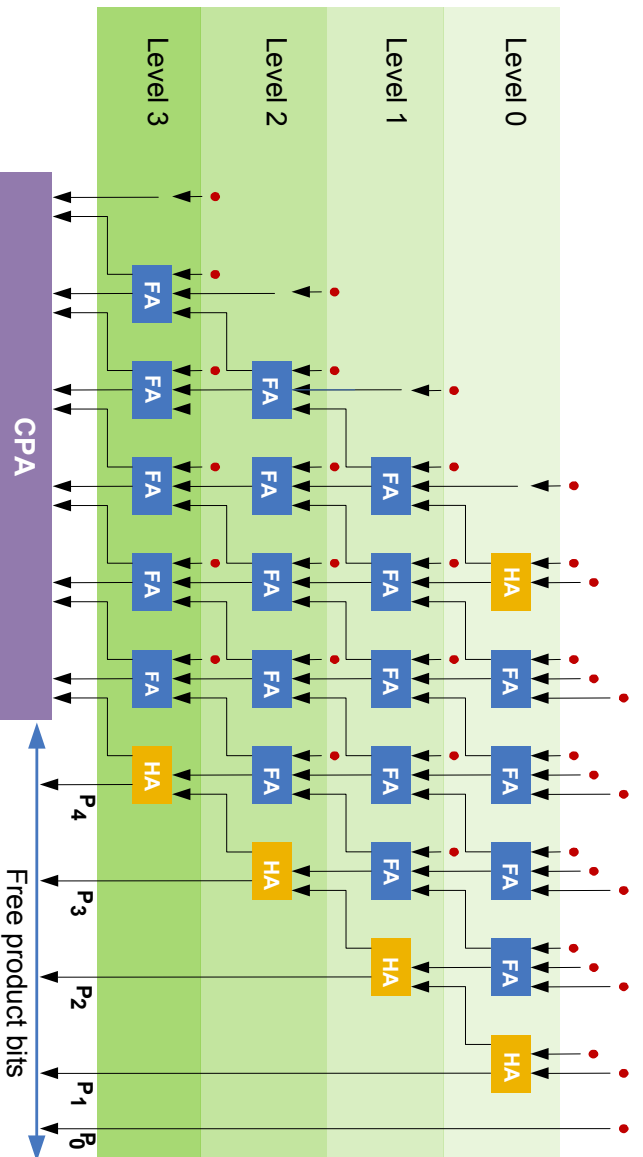
- The partial products are divided into 2 equal size groups
- The carry save reduction scheme is applied on both the groups simultaneously

- This results into two partial product layers in each group
- The four layers are then reduced using Carry Save

Reduction

- The last two layers are added using any CPA

Carry Save Reduction Scheme Layout for a 6x6 Multiplier



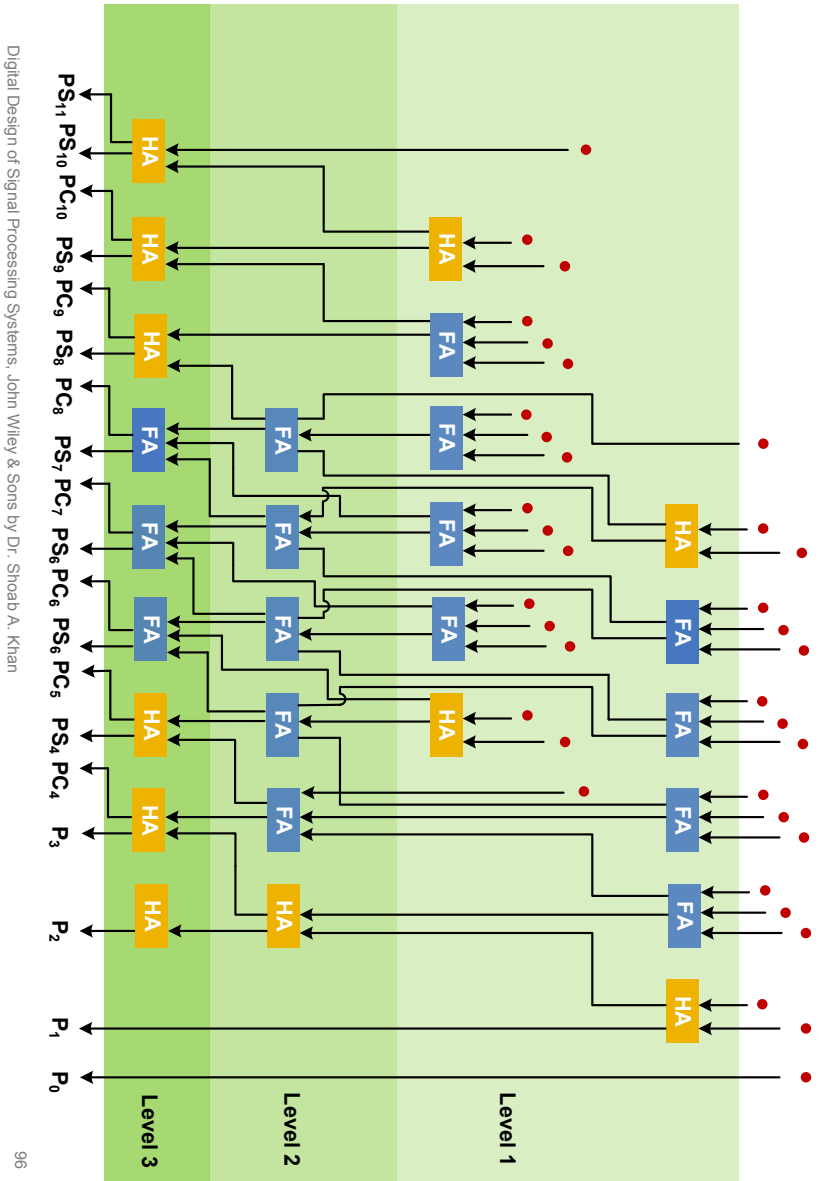
Wallace Tree Multipliers

- Make group of threes and apply CSA reduction in parallel
- Each CSA layer produces two rows
- These rows then, with other rows from other partial product groups, form a new reduced matrix
- Iteratively apply Wallace reduction on the new generated matrix
- This process continues until only two rows are left
- The final rows are added together for the final product

Wallace Tree Multipliers

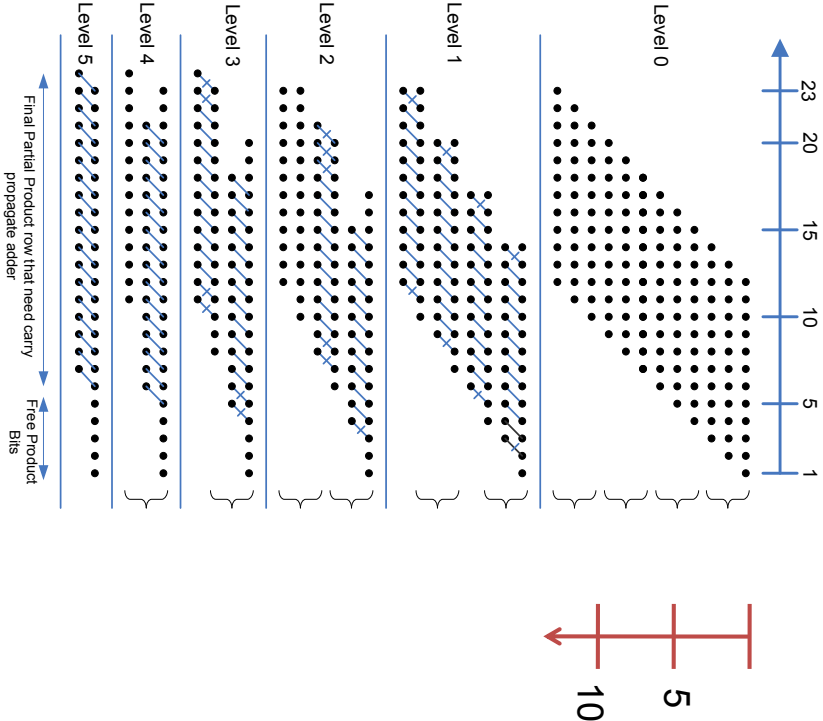
- One of the most commonly used multiplier architecture
- It is log time array multiplier
- The number of adder levels increases logarithmically as the partial product rows increase

Wallace Reduction layout for a 6x6 array of PPs



Digital Design of Signal Processing Systems, John Wiley & Sons by Dr. Shoaib A. Khan

Wallace Reduction Tree applied on 12 PPs



Dada Reduction

- Minimizes the number of HAs and FAs
- Reduction considers each column separately
- Reduces the number of dots in each column to the maximum number of layers in the next level in Wallace Reduction Table

Dada Reduction uses the Wallace Reduction Table

Adder Levels in Wallace Tree Reduction Scheme

Number of partial Products	Number of full adder Levels
3	1
4	2
$5 \leq n \leq 6$	3
$7 \leq n \leq 9$	4
$10 \leq n \leq 13$	5
$14 \leq n \leq 19$	6
$20 \leq n \leq 28$	7
$29 \leq n \leq 42$	8
$43 \leq n \leq 63$	9

As the string property is applied on three bits, there are following eight possibilities:

$2^1=2$	$2^0=1$			
0	0	0	0	
0	0	1	1	
0	1	0	1	
0	1	1	0	2
1	0	0	0	-2
1	0	1	1	-1
1	1	0	0	-1
1	1	1	1	0

Dadda reduction levels for reducing eight PPs to two

